

Computer Arkitektur og Operativ Systemer Assembler 2

Forelæsning 4
Brian Nielsen

*Credits to
Randy Bryant & Dave O'Hallaron (CMU)*

Opened: Sunday, 18 February 2024, 11:30 AM ... **Closes:** Friday, 29 August 2025, 12:30 PM



Quiz

Demo Eksamen i "quizz format" (demo) (v2) [↗](#)

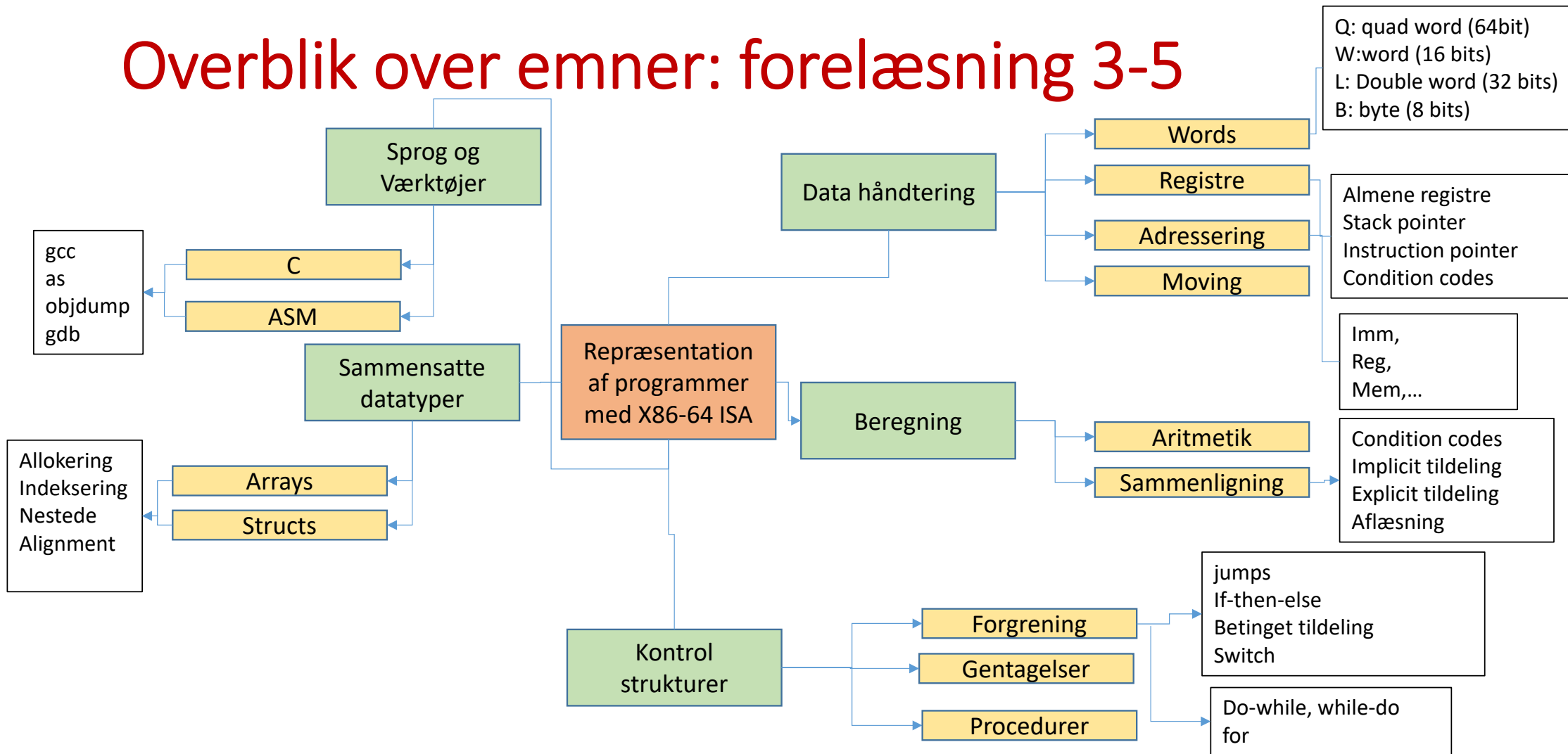
PopQuizz

- Hvad er semantikken af “særlig vigtig” (øvelser) ?

Vælg alle gyldige muligheder.

- a) $[[\text{“særlig vigtig”}]] =_{\text{def}} \text{ligegyldige, ignorér}$
- b) $[[\text{“særlig vigtig”}]] =_{\text{def}} \text{for de særligt interesserede}$
- c) $[[\text{“særlig vigtig”}]] =_{\text{def}} \text{kun disse er relevante for eksamen}$
- d) $[[\text{“særlig vigtig”}]] =_{\text{def}} \text{hvis vi er helt presset for tid, kan vi vente med disse til senere}$
- e) $[[\text{“særlig vigtig”}]] =_{\text{def}} \text{viser en pointe eller konstruktion vi skal være særlig opmærksom på}$

Overblik over emner: forelæsning 3-5



Idag

- Hvordan repræsenteres kontrol-strukturer på maskine-niveau ?
 - `if (c) { then; }`
 - `if (c) { then; } { else; }`
 - `while (c) { body; }`
 - `do { body; } while (b)`
 - `for (i; c; u) { body; }`
 - `switch`, Skippes med mindre du er særligt interesseret (jump-tables)
- Hvilke oversættelses-strategier og -skabeloner anvender compileren (backend delen)?
- Hvordan repræsenteres og tilgås sammensatte datatyper
 - Arrays?
 - Structs ?

Hvordan evalueres betingelser?

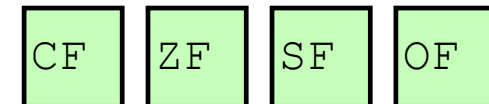
- Hvordan repræsenteres kontrol-strukturer på maskine-niveau ?

- Selektion: fx: `if (c) { then; } { else; }`
- Iteration, fx: `while (c) { }`

1. Evaluering af betingelser **c**: Condition Code
Register indeholder status på seneste "udregning"

Carry (Mente)-flag
Zero (Nul)-flag
Sign (Fortegns)-flag
Overflow (Overløbs)-flag

Betingelsesflag
Condition codes



- Sættes af instruktioner, som er klassificeret som "aritmetiske", fx *add*, *sub*, *mult*, *sar*, *shl*, ..
- Sættes af instruktioner til sammenligning:
 - **cmp src1,src2**: beregner **src2-src1**, ændrer KUN CC register; resultat smides væk
IDE: `src2-src1 == 0 : src1 == src2 ZF?`
`src2-src1 < 0 : src1 > src2 SF?` //(og der ikke sker overløb)
 - **tst src1,src2**: anvender and istedet for minus.

2. (En del af "processor status word" (PSW aka FLAGS) register, som indeholder andre status flag)

Hvordan laves betinget forgrening

1. Hops / jumps

1. Ubetingede: **jmp** <adresse>
2. Betingede hop: **JNE** <adresse>

Hopper hvis : **~ZF**

Fortolkning : Not Equal / Not Zero

Ændrer:

- Program tæller (program counter)
a.k.a.
- Instruction pointer register

`%rip`

2. Oversættelses-skabeloner (med "goto" kode)

- Som mellemtrin fra c-kode til assembler
- Genkende en anvendt skabelon ved "dekompilering" fra ASM til C

C kode

```
long pcount_do
(unsigned long x) {
    long result = 0;
    do {
        result += x & 0x1;
        x >>= 1;
    } while (x);
    return result;
}
```

Goto Version

```
long pcount_goto
(unsigned long x) {
    long result = 0;
loop:
    result += x & 0x1;
    x >>= 1;
    if(x) goto loop;
    return result;
}
```

ASM

```
##%rdi=x
##%rax=result
movl    $0, %eax    # result = 0
.L2:
    # loop:
    movq    %rdi, %rdx # t= x
    andl    $1,%edx   # t= x & 0x1
    addq    %rdx, %rax # result += t
    shrq    %rdi      # x >>= 1
    jne     .L2        # if (x) goto loop
ret
```

Quizz: Beregningsspor kontrol -1

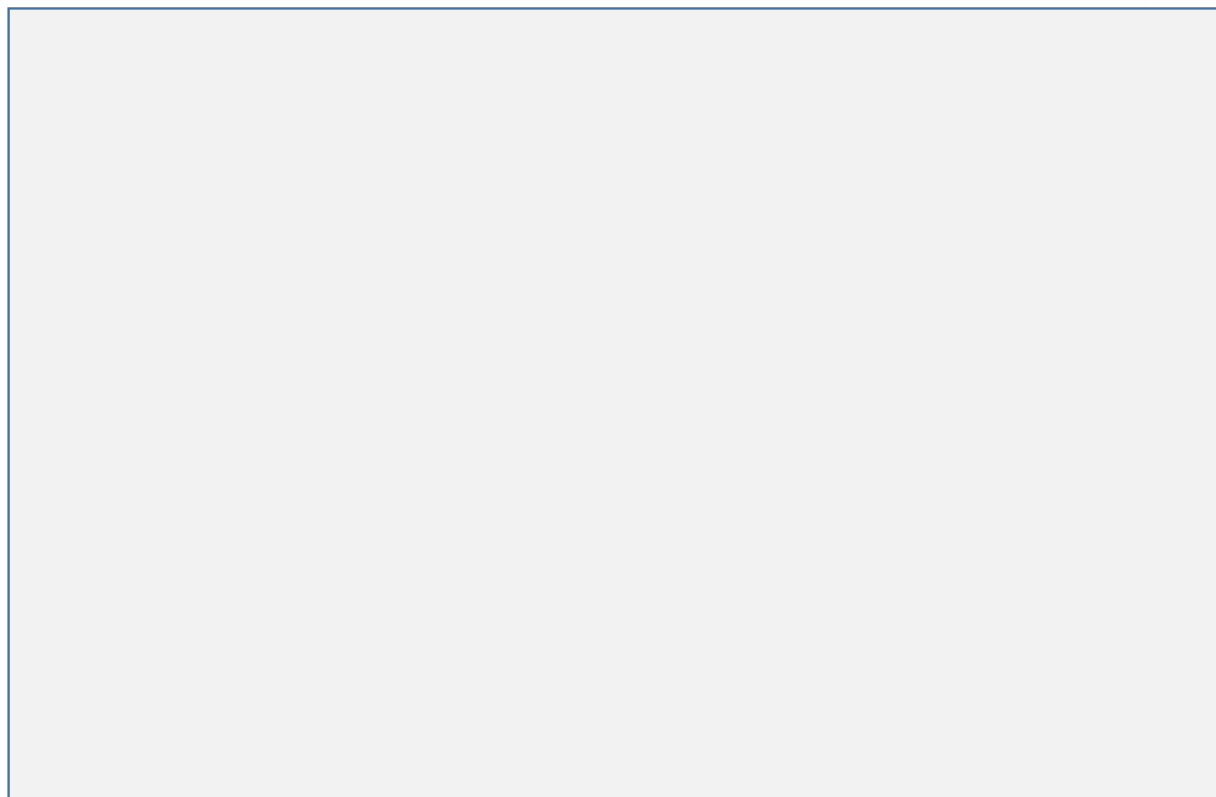
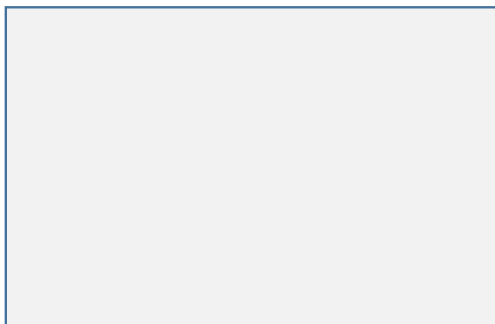
I det følgende er vist et fragment af et X86-64 assembly program, som gcc har oversat fra et lille C-program.

Lav et beregningsspor (execution trace) af funktionen ved at udfylde nedenstående tabel givet start tilstanden givet i første linie. Ved start har %rdi værdien 4. Du kan antage, at der ikke sker overflow.

Dvs. angiv effekten på processorens tilstanden ved at hånd-eksekvere instruktionerne én efter én. I linie n skal du angive den tilstand, der gælder inden instruktionen udføres; dens resultat (ændring af register-værdier) skal dermed stå i linien under: n+1. Hvis en værdi ikke kan udledes, bedes du angive dette med et "-"

0000000000401136 <func>:

```
401136: mov    %rdi,%rax
401139: jmp    40113f <func+0x9>
40113b: sub    $0x2,%rax
40113f: cmp    $0x1,%rax
401143: jg     40113b <func+0x5>
401145: ret
```



Quizz: Beregningsspor kontrol -2

Nedenfor er vist et stykke assembler, som er oversat af gcc fra en C-funktion.

```
401136: mov %rdi,%rax
401139: jmp 40113f
40113b: sub $0x2,%rax
40113f: cmp $0x1,%rax
401143: jg 40113b
401145: ret
```

Fra hvilken af nedenstående C-funktioner stammer assembler koden:

C-funktioner

func0	func1	func2	func3
<pre>long func0(long x){ while(x>=2){ x=x-2; } return x; }</pre>	<pre>long func1(long x){ do{ x=x-2; } while(x>=2); return x; }</pre>	<pre>long func2(long x){ do{ x=x-2; } while(x>1); return x; }</pre>	<pre>long func3(long x){ x=x-2; if(x>1) return x; else x=1; return x; }</pre>

Quizz: Beregningsspor kontrol -3

Koden nedenfor anvender en kendt kontrol struktur, hvilken?

Listing 2: Disassembleret oversat C program.

```
1 det :  
2 0x4005b0 <+0>: movq    0x8(%rsi),%rax  
3 0x4005b4 <+4>: imul    (%rdi),%rax  
4 0x4005b8 <+8>: movq    (%rsi),%rdx  
5 0x4005bb <+11>: imul    0x8(%rdi),%rdx  
6 0x4005c0 <+16>: subq    %rdx,%rax  
7 0x4005c3 <+19>: js      0x4005c7 <det+23>  
8 0x4005c5 <+21>: retq  
9 0x4005c7 <+23>: negq    %rax  
10 0x4005ca <+26>: jmp     0x4005c5 <det+21>
```

- a. switch flervejs selektion
- b. while løkke
- c. for-løkke
- d. do-while løkke
- e. if-then selektion
- f. procedurekald

[Clear my choice](#)

Betinget tildeling

- Nogle betingede tildelinger og nogle simple if-then-else kan med fordel oversættes uden betinget forgrening

C kode

```
val = x > y ? x - y : y - x;
```

```
val = Test ? Then_Expr : Else_Expr;
```

- Conditional Move instruktioner **cmovX Src, Dest**
 - Instruktionen understøtter: **if (Test) Dest ← Src**
 - **cmovle %rdx, %rax**
 - **cmovg %rdx, %rax**
- GCC forsøger at anvende dem
 - Men kun når den ved at det er sikkert
- Hvorfor?
 - Betinget forgrening via betingede jumps er meget forstyrrende for instruktions-flow igennem processorens pipeline ("Branch-prediction")
 - Conditional move kan ofte undgå ændring af kontrol-flow

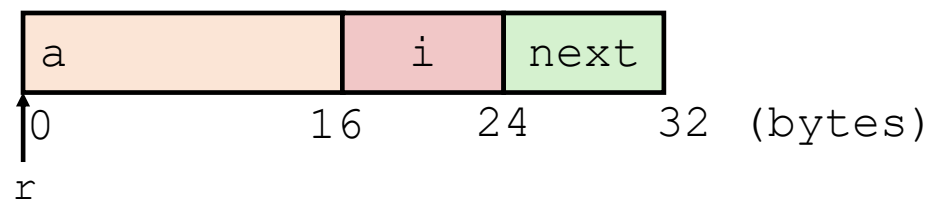
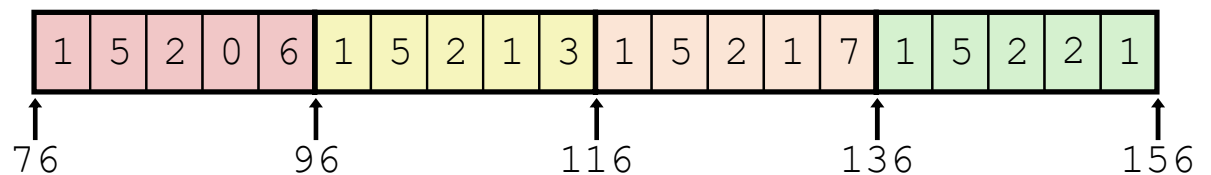
NB: beregner både "then", og "else" udtryk
Udvælger den version, der skal bruges

Hvordan layouts sammensatte datatyper?

- Arrays og structs findes ikke på maskine-niveau
- Hvordan udlægges de som stykker hukommelse (i ram)?
 - Arrays i flere dimensioner? → Rækkevist!
 - Heterogene datatyper? → Erklærings-rækkefølge + alignment
- Hvordan indekseres de? → Adresse-beregning

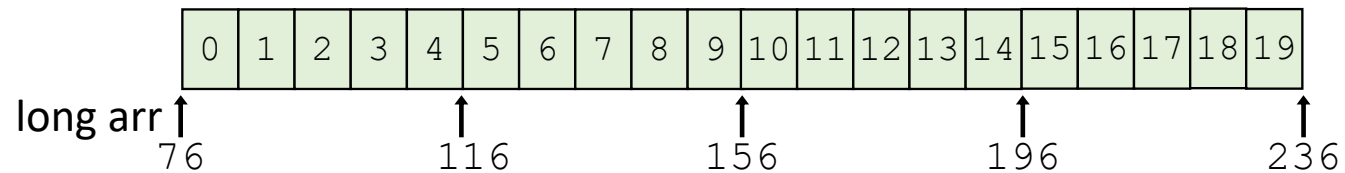
```
int pgh[4][5]=  
  {{1, 5, 2, 0, 6},  
   {1, 5, 2, 1, 3 },  
   {1, 5, 2, 1, 7 },  
   {1, 5, 2, 2, 1 }};
```

```
struct rec {  
  int a[4];  
  size_t i;  
  struct rec *next;  
};
```



Arrays, typer, og mov instruktionen?

```
long arr[20]= {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 ,...,19};  
long x=arr[5];  
long* y=&arr[5];
```

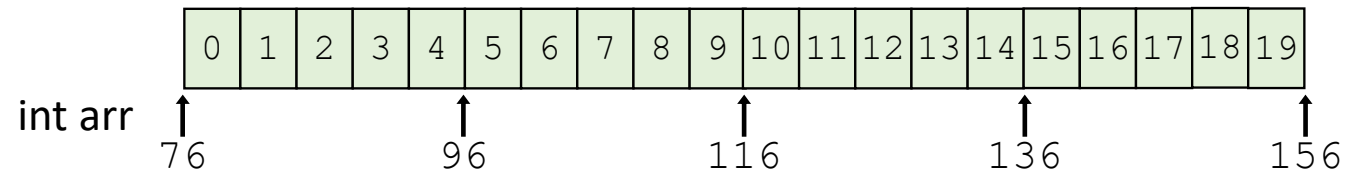


“arr” og “x” er LONG type:

```
##rbx = 76, ##rsi = 5      #base, array index 5  
movq (%rbx,%rsi,8), %rdx   ##rdx = mem8[rbx+rsi*8] = mem8[116]=5  
leaq (%rbx,%rsi,8), %rcx   ##rcx= rbx+rsi*8 =116
```

Arrays, typer, og mov instruktionen?

```
int arr[20]= {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 ,...,19};  
int x=arr[5];  
int* y=&arr[5];
```

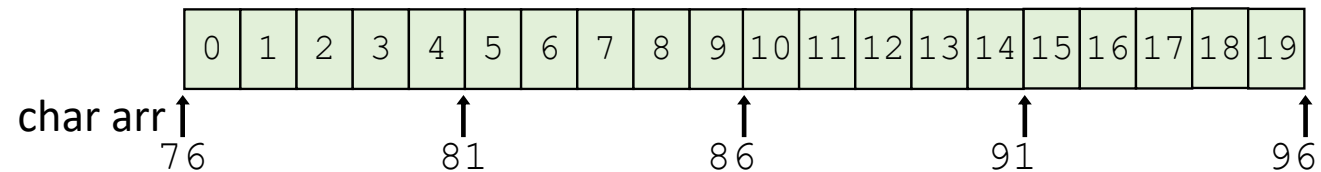


“arr” og “x” er INT type:

```
##rbx = 76, ##rsi = 5      #base, array index 5  
movl (%rbx,%rsi,4), %edx   #edx = mem4[rbx+rsi*4] = mem4[96]=5, (%rdx=5 pr 64-bit finte, implicit zero extended from long word (int) to quad)  
leaq (%rbx,%rsi,4), %rcx   #%rcx= rbx+rsi*4 =96
```

Arrays, typer, og mov instruktionen?

```
char arr[20]= {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 ,...,19};  
char x=arr[5];  
char* y=&arr[5];
```

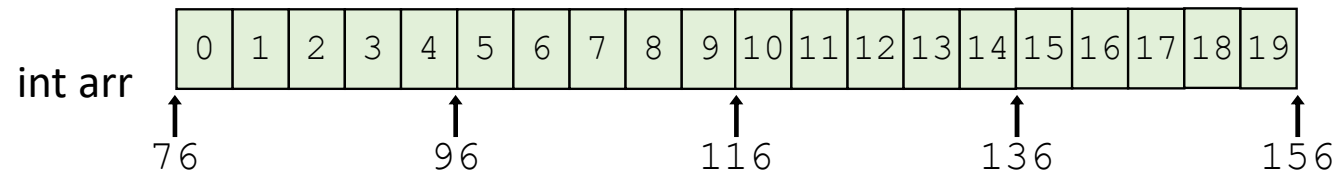


“arr” og “x” er CHAR type:

```
%%rbx = 76, %%rsi = 5      #base, array index 5  
movb (%rbx,%rsi,1), %dl    %%dl = mem1[rbx+rsi*1] = mem1[81]=5  
leaq (%rbx,%rsi,1), %rcx   %%rcx= rbx+rsi*1 = 81
```

Arrays, typer, og mov instruktionen?

```
int arr[20]= {0, 1, 2, 3, 4, 5,6, 7, 8, 9, 10 ,...,19};  
long x=arr[5];  
int* y=&arr[5];
```



“arr” er INT type:

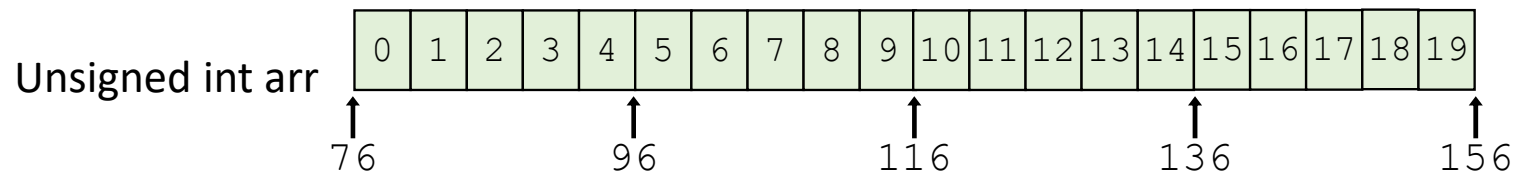
`##rbx = 76, ##rsi = 5` *#base, array index 5*

`movslq (%rbx,%rsi,4), %rdx` *#rdx = mem₄[rbx+rsi*4] = mem₄[96]=5, rdx=5, sign extended from long word (int) to quad*

`leaq (%rbx,%rsi,4), %rcx` *##rcx= rbx+rsi*4 =96*

Arrays, typer, og mov instruktionen?

```
unsigned int arr[20]= {0, 1, 2, 3, 4, 5,6, 7, 8, 9, 10 ,...,19};  
long x=arr[5];  
Unsigned int* y=&arr[5];
```

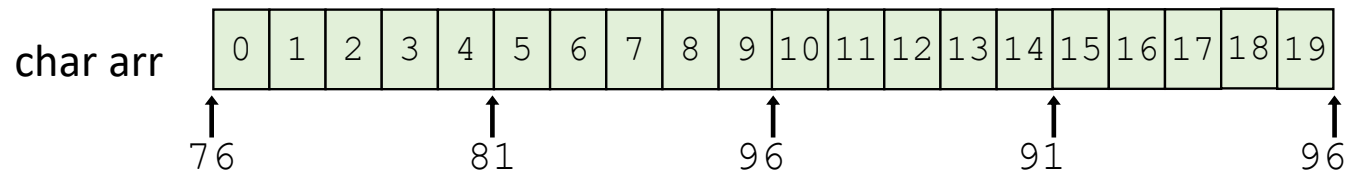


“arr” er UNSIGNED INT type:

<code>##rbx = 76, ##rsi = 5</code>	<i>#base, array index 5</i>
<code>movl (%rbx,%rsi,4), %edx</code>	<i>##edx = mem₄[rbx+rsi*4] = mem₄[96]=5, %rdx=5 pr 64-bit finte, implicit zero extended from long word (int) to quad</i>
<code>leaq (%rbx,%rsi,4), %rcx</code>	<i>##rcx= rbx+rsi*4 = 96</i>

Arrays, typer, og mov instruktionen?

```
char arr[20]= {0, 1, 2, 3, 4, 5,6, 7, 8, 9, 10 ,...,19};  
long x=arr[5];  
char* y=&arr[5];
```



“arr” er CHAR type:

`##rbx = 76, ##rsi = 5` *#base, array index 5*

`movsbq (%rbx,%rsi,1), %rdx` *##rdx = mem₁[rbx+rsi*1] = mem₁[81]=5, sign extended from byte to quad*

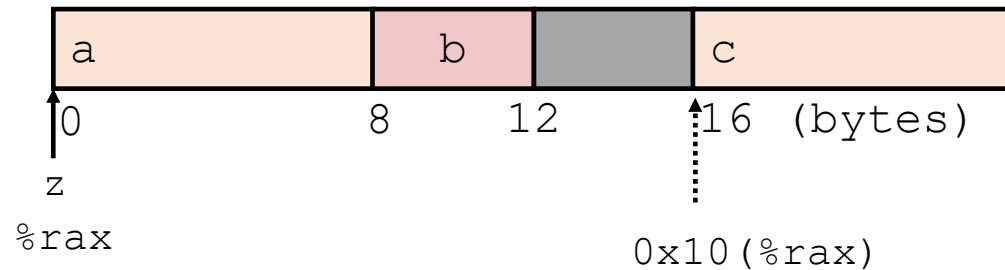
`leaq (%rbx,%rsi,1), %rcx` *##rcx= rbx+rsi*1 = 81*

Structs og MOV

- Eksempel brug af “Displacement”

```
struct EX {  
    long a  
    int b;  
    long c;  
}*z;  
z=malloc(...)  
z->c=3;
```

```
#z in %rax  
movq    $0x3, 0x10(%rax)
```



- NB: Alignment, padding, og optimeringstrick

Øvelserne

- Fra ASM til goto-kode (de-kompilering af if)
 - Oversættelsesstrategi for if
 - Arrays layout & indeksering
-
- Challenge 3: Injicér objekt-kode for at forhindre atomkrig
 - Challenge 4: ASM indlejret i C!!



War Games '83

|Hjælpelærer|=Max - Brug dem!

